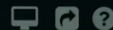
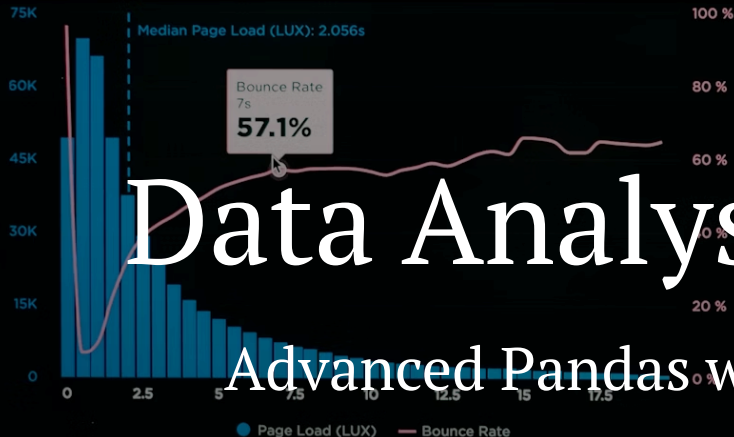


USERS: LAST 7 DAYS USING MEDIAN ▾



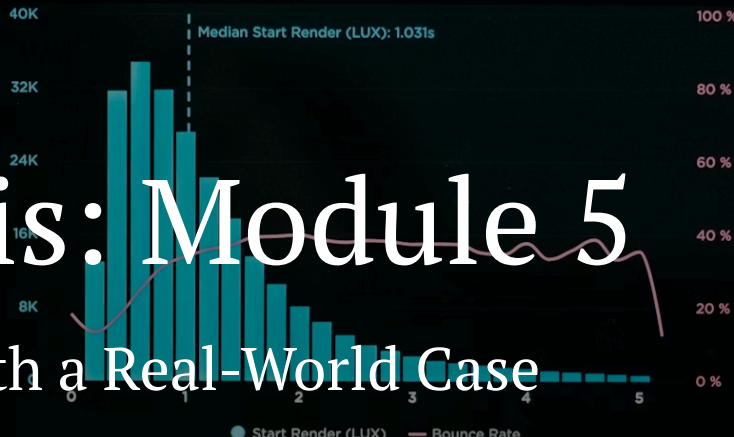
LOAD TIME VS BOUNCE RATE

⚙️ OPTIONS



START RENDER VS BOUNCE RATE

⚙️ OPTIONS

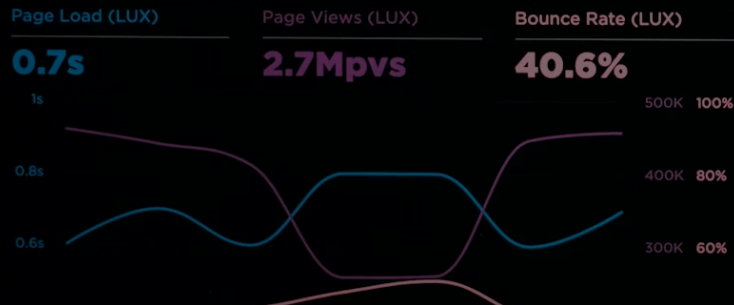


Data Analysis: Module 5

Advanced Pandas with a Real-World Case

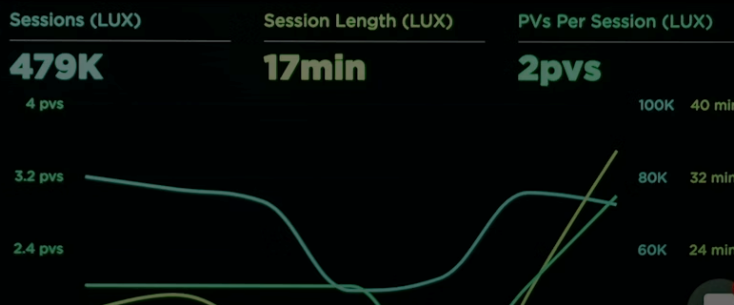
PAGE VIEWS VS ONLOAD

⚙️ OPTIONS



SESSIONS

⚙️ OPTIONS



1. Goal of This Lecture

In the basics lecture, you learned how to create, inspect, and filter DataFrames.

In this lecture, we move into real analysis work:

1. Clean messy business data
2. Group and summarize by category
3. Merge data from multiple tables
4. Work with dates and time series
5. Answer a real business question with Pandas

2. Real-World Problem

Online Store Sales Analysis

You work for an e-commerce company.

You receive:

- an `orders` table
- a `customers` table

You need to answer:

- Which product category makes the most revenue?
- Which city has the highest average order value?
- Which days had unusually high sales?
- Which customer segments should marketing focus on?

3. Build the Dataset

```
1  import pandas as pd
2
3  orders = pd.DataFrame({
4      'order_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007],
5      'customer_id': [1, 2, 1, 3, 4, 2, 5],
6      'category': ['Electronics', 'Books', 'Electronics',
7                  'Home', 'Books', 'Home', 'Electronics'],
8      'amount': [1200, 25, 900, 150, 30, 220, 1600],
9      'order_date': ['2026-04-01', '2026-04-01', '2026-04-02',
10                   '2026-04-02', '2026-04-03', '2026-04-03', '2026-04-04']
11  })
12
13  customers = pd.DataFrame({
14      'customer_id': [1, 2, 3, 4, 5],
15      'city': ['Toronto', 'Seoul', 'Toronto', 'Vancouver', 'Seoul'],
16      'segment': ['Business', 'Student', 'Business', 'Student', 'Business']
17  })
```

4. First Step: Inspect the Data

```
1 print(orders.head())
2 print(orders.info())
3 print(orders.describe())
```

```
1 print(customers.head())
2 print(customers.info())
```

Rule

Before analysis, check:

- shape
- column names
- data types
- missing values

5. Clean the Date Column

Dates are often loaded as strings.

```
1 orders['order_date'] = pd.to_datetime(orders['order_date'])
2 print(orders.dtypes)
```

Why it matters

Once a column is datetime, Pandas can group by day, month, weekday, and more.

6. Column Creation

Feature engineering is common in real work.

```
1  orders['tax'] = orders['amount'] * 0.13
2  orders['final_amount'] = orders['amount'] + orders['tax']
3  orders['is_large_order'] = orders['amount'] >= 500
4
5  print(orders.head())
```

This is useful for

- reporting
- pricing analysis
- fraud detection
- machine learning features

7. GroupBy: The Core Skill

Group data by category and summarize:

```
1  category_summary = orders.groupby('category')['amount'].agg(  
2      ['count', 'sum', 'mean', 'max']  
3  )  
4  
5  print(category_summary)
```

Think of `groupby` as

split -> apply -> combine

8. Grouping by More Than One Column

```
1 city_category = orders.groupby(['category', 'customer_id'])['amount'].sum()
2 print(city_category)
```

```
1 segment_summary = orders.groupby('customer_id')['amount'].sum()
2 print(segment_summary)
```

This is useful when analysis depends on more than one business dimension.

9. Merge Tables

Real datasets are often split across multiple files or database tables.

```
1 full_data = orders.merge(customers, on='customer_id', how='left')
2 print(full_data)
```

Why `merge()` matters

Without it, most business analysis is impossible.

You often need transaction data plus customer or product metadata.

10. Analyze by Customer Segment

```
1 segment_revenue = full_data.groupby('segment')['amount'].sum()
2 segment_avg = full_data.groupby('segment')['amount'].mean()
3
4 print(segment_revenue)
5 print(segment_avg)
```

Business question

Does one segment buy more often, or just place larger orders?

11. Analyze by City

```
1 city_summary = full_data.groupby('city')['amount'].agg(  
2     ['count', 'sum', 'mean']  
3 ).sort_values('sum', ascending=False)  
4  
5 print(city_summary)
```

Real use

This helps teams decide:

- where to advertise
- where to open warehouses
- where to offer promotions

12. Missing Data Strategy

Suppose one amount is missing:

```
1 orders_with_nan = orders.copy()
2 orders_with_nan.loc[3, 'amount'] = None
3
4 print(orders_with_nan.isna().sum())
```

Common choices:

```
1 print(orders_with_nan.dropna())
2 print(orders_with_nan.fillna(0))
3 print(orders_with_nan['amount'].fillna(orders_with_nan['amount'].mean()))
```

13. Sorting and Ranking

```
1 top_orders = full_data.sort_values('amount', ascending=False)
2 print(top_orders[['order_id', 'city', 'category', 'amount']])
```

```
1 category_totals = full_data.groupby('category')['amount'].sum()
2 print(category_totals.sort_values(ascending=False))
```

Ranking is common in reporting dashboards.

14. Time Series Analysis

Daily revenue:

```
1 daily_sales = orders.groupby('order_date')['amount'].sum()
2 print(daily_sales)
```

Add rolling average:

```
1 rolling_sales = daily_sales.rolling(2).mean()
2 print(rolling_sales)
```

Why this matters

Time trends matter more than isolated rows.

15. Detect Busy Days

```
1  daily_sales = orders.groupby('order_date')['amount'].sum()
2
3  mean_sales = daily_sales.mean()
4  std_sales = daily_sales.std()
5
6  high_days = daily_sales[daily_sales > mean_sales + std_sales]
7  print(high_days)
```

This is a simple way to flag unusual sales spikes.

16. Pivot Tables

Pivot tables create report-friendly summaries.

```
1  pivot = pd.pivot_table(  
2      full_data,  
3      values='amount',  
4      index='city',  
5      columns='category',  
6      aggfunc='sum',  
7      fill_value=0  
8  )  
9  
10 print(pivot)
```

Useful for

- management reports
- finance summaries
- dashboard exports

17. Real Interpretation

From this dataset, we can answer:

1. Electronics generates the highest revenue
2. Some cities have fewer orders but higher average order values
3. Sales vary by day, and spikes can be detected quickly
4. Customer metadata changes how we interpret raw transactions

This is the value of Pandas

It turns tables into business decisions.

18. Mini Exercise 1

Using `orders` :

1. Find total revenue
2. Find the average order amount
3. Find the largest single order

```
1  # Write your code here
```

19. Mini Exercise 2

Using `full_data` after merge:

1. Find total revenue by city
2. Find average order amount by segment
3. Sort categories by total revenue

20. Mini Exercise 3

Create a new column called `discount_band` :

- `"High"` if `amount >= 1000`
- `"Medium"` if `amount >= 100`
- `"Low"` otherwise

Hint:

```
1 def label_order(x):  
2     ...
```

Apply it to the `amount` column.

21. Challenge Exercise

Marketing Budget Decision

A marketing manager can only focus on one customer segment next month.

Your task:

1. Merge the two tables
2. Compare total revenue by segment
3. Compare average order value by segment
4. Decide which segment should be prioritized
5. Explain your reasoning in 2-3 sentences

Real work is not only code

You must support a decision with evidence.

22. Exercise Solutions

```
1 print(orders['amount'].sum())
2 print(orders['amount'].mean())
3 print(orders['amount'].max())
```

```
1 full_data = orders.merge(customers, on='customer_id', how='left')
2 print(full_data.groupby('city')['amount'].sum())
3 print(full_data.groupby('segment')['amount'].mean())
4 print(full_data.groupby('category')['amount'].sum().sort_values(ascending=False))
```

23. More Solution Ideas

```
1  def label_order(x):
2      if x >= 1000:
3          return 'High'
4      elif x >= 100:
5          return 'Medium'
6      else:
7          return 'Low'
8
9  orders['discount_band'] = orders['amount'].apply(label_order)
10 print(orders[['amount', 'discount_band']])
```


24. Summary

In advanced Pandas work, the core ideas are:

- inspect before analysis
- clean types and missing data
- create useful columns
- group and aggregate
- merge related tables
- analyze change over time

If you can do these well, you are using Pandas for real analysis work.

End

Reference Sources:

- Pandas Official Getting Started
- Pandas User Guide: Group By
- Pandas User Guide: Merge, Join, and Concatenate